

SOA CAN Module Architecture and Design

This document describes the current SOA CAN module architecture in the M-Core project. The focus is the active software path in the current code, not a future target architecture.

1. System Overview

The SOA CAN module is responsible for cyclic transmission and reception on FlexCAN0. In the current project, the active path is:

- PreOS initialization in `main()` completes FlexCAN controller startup and RX MB configuration.
- Runtime processing is driven by the unified 10 ms FreeRTOS task.
- DBC conversion is handled by the generated `SOA_CANdbc_Generated` layer.
- The current active cyclic function is `AINFC_Can_Cyclic_10ms()`.

`SoaAdapter_Init()` is already called during initialization, but `SoaAdapter_Main()` is not scheduled in the current runtime path.

2. Module Architecture

The current CAN module can be understood as three layers:

1. Application Data Layer

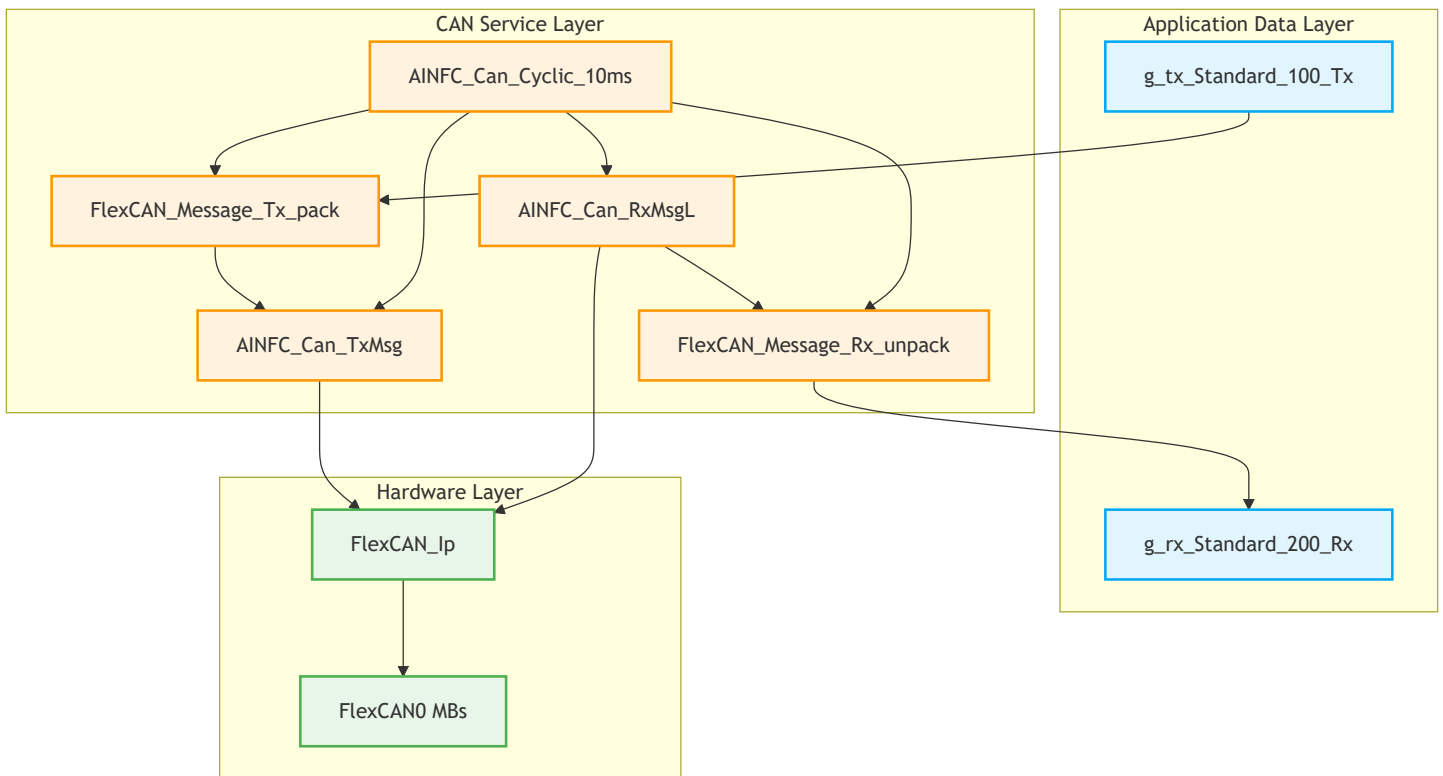
- Uses the generated global structures `g_tx_Standard_100_Tx` and `g_rx_Standard_200_Rx`.

2. CAN Service Layer

- `AINFC_Can_Cyclic_10ms()` drives cyclic RX/TX.
- `FlexCAN_Message_Tx_pack()` and `FlexCAN_Message_Rx_unpack()` connect DBC conversion with CAN send/receive.
- `AINFC_Can_TxMsg()` and `AINFC_Can_RxMsgL()` provide the direct FlexCAN wrapper interface.

3. Hardware Layer

- NXP RTD `FlexCAN_Ip` APIs access FlexCAN0 message buffers.



3. Runtime Flow

3.1 Initialization Flow

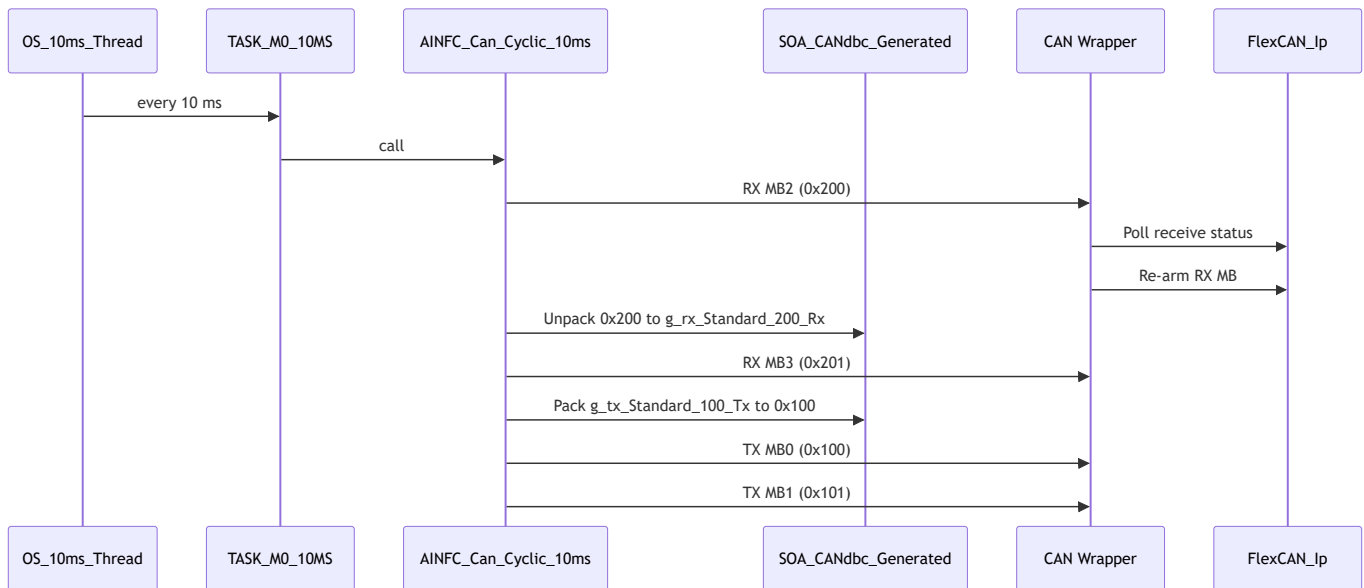
The CAN-related startup sequence in `main()` is:

1. `FlexCAN_Ip_Init()`
2. `FlexCAN_Ip_SetStartMode()`
3. `FlexCAN_Process_Init()`
4. `PICC_PreOS_Init()`
5. `SoaAdapter_Init()`
6. `OsTask_Creation_All()`

This means FlexCAN0 is fully initialized before the scheduler starts.

3.2 10ms Cyclic Flow

At runtime, the CAN module is executed from `TASK_M0_10MS()` inside the unified `OS_10ms` thread.



The current code executes this path every 10 ms.

4. Message and MB Mapping

Direction	MB	CAN ID	Usage
TX	MB0	0x100	DBC TX message, source is g_tx_Standard_100_Tx
TX	MB1	0x101	Raw debug frame
RX	MB2	0x200	DBC RX message, target is g_rx_Standard_200_Rx
RX	MB3	0x201	Raw RX frame

The current generated DBC layer only covers:

- 0x100 TX
- 0x200 RX

0x101 and 0x201 are not DBC messages in the current implementation.

5. Key Components and Files

- src/Ecum_Init_Main/EcuM_main_init.c
 - CAN startup entry.
 - Calls FlexCAN initialization and FlexCAN_Process_Init() .
- src/Task/Ostask_main.c
 - Runtime scheduling entry.

- Calls `AINFC_Can_Cyclic_10ms()` from `TASK_M0_10MS()` .
- `COM/FlexCAN_Ip/FlexCAN_Ip_main.c / .h`
 - Main CAN business logic.
 - Provides MB mapping, RX/TX wrapper functions and cyclic processing.
- `SWC/Soa_Adapter/SOA_CANdbc_Generated.c / .h`
 - Generated DBC pack/unpack layer.
 - Provides the global TX/RX message structures.
- `SWC/Soa_Adapter/Soa_Adapter.c / .h`
 - Current SOA registration layer.
 - `SoaAdapter_Init()` is active, but `SoaAdapter_Main()` is not part of the current runtime flow.

6. Current Implementation Notes

- The active CAN runtime path is `TASK_M0_10MS()` -> `AINFC_Can_Cyclic_10ms()` .
- `SoaAdapter_Main()` is not scheduled in the current code.
- The generated DBC metadata defines `0x100` and `0x200` as 100 ms messages, but the current runtime path executes every 10 ms.
- `0x100` is generated as a 2-byte DBC message, but the current FlexCAN TX data info is configured with DLC 8.
- `FlexCAN_GetBusDiagStatus()` exists as a diagnostic interface, but it is not yet integrated into the periodic runtime path.